

Programação Orientada a Objetos Aula 09

João Choma Neto
joao.choma@gmail.com

Chamada está em

- <https://github.com/JoaoChoma/POO>

Redefinição de métodos

- Redefinição de métodos, também conhecida como sobrescrita de métodos, é um conceito em Programação Orientada a Objetos (POO) que permite que uma classe filha substitua ou modifique a implementação de um método herdado de sua classe pai
- Isso significa que a classe filha tem uma implementação diferente do método da classe pai, mas com a mesma assinatura (nome, parâmetros e tipo de retorno)

Redefinição de métodos

- Quando o método é chamado para um objeto da classe filha, a implementação da classe filha é executada em vez da implementação da classe pai
- A redefinição de métodos é uma técnica importante na POO, pois permite que as classes filhas forneçam uma funcionalidade específica que complementa ou substitui a funcionalidade herdada da classe pai

Análise

- Suponha que temos uma classe `Animal` com um método chamado `fazerSom()`, que imprime na tela "O animal faz um som genérico"
- Vamos criar uma classe `Cachorro` que herda da classe `Animal` e redefine o método `fazerSom()` para imprimir "O cachorro late".

Exemplo

```
class Animal {  
    public void fazerSom() {  
        System.out.println("O animal faz um som genérico");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void fazerSom() {  
        System.out.println("O cachorro late");  
    }  
}
```

Exemplo

- Na classe Cachorro, estamos usando a anotação **@Override** para indicar que estamos redefinindo o método **fazerSom()** da classe pai Animal

Execução

```
public class TesteAnimais {  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        Cachorro cachorro = new Cachorro();  
  
        animal.fazerSom(); // imprime "O animal faz um som genérico"  
        cachorro.fazerSom(); // imprime "O cachorro late"  
    }  
}
```


Explicação

- O método **fazerSom()** da classe **Cachorro** foi redefinido e está sendo executado no objeto da classe Cachorro

Novo exemplo - Redefinição

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public void fazerAniversario() {  
        this.idade++;  
        System.out.println("Parabéns, " + this.nome + "! Você agora tem " + this.idade + " anos.");  
    }  
}
```

Explicação

- A classe **Pessoa** possui um método **fazerAniversario** que simplesmente incrementa a idade da pessoa e exibe uma mensagem

```
public class Funcionario extends Pessoa {  
    private double salario;
```

```
    public Funcionario(String nome, int idade,  
double salario) {  
        super(nome, idade);  
        this.salario = salario;  
    }
```

```
    public void aumentarSalario(double  
percentual) {  
        double aumento = this.salario *  
percentual / 100.0;  
        this.salario += aumento;  
        System.out.println("O salário de " +  
this.nome + " foi aumentado em " +  
percentual + "%". Novo salário: R$ " +
```

```
this.salario);  
    }
```

```
@Override
```

```
    public void fazerAniversario() {  
        super.fazerAniversario();  
        if (this.idade % 5 == 0) {  
            double bonus = this.salario * 0.1;  
            this.salario += bonus;  
            System.out.println("Parabéns, " +  
this.nome + "! Você agora tem " + this.idade  
+ " anos e ganhou um bônus de R$ " + bonus  
+ ".");  
        }  
    }  
}
```

Explicação

- A classe **Funcionario** estende a classe **Pessoa** e possui um atributo **salario** e um método **aumentarSalario**, que aumenta o salário do funcionário de acordo com um percentual informado
- A classe **Funcionario** redefine o método **fazerAniversario** para, além de incrementar a idade e exibir uma mensagem, verificar se a idade é múltipla de 5 e, nesse caso, conceder um bônus ao funcionário

Atividade redefinição

- Implemente o seguinte problema:
- Uma classe **Pessoa** que possui um método **fazerAniversario** que simplesmente incrementa a idade da pessoa e exibe uma mensagem

Atividade redefinição

- Implemente uma classe **Funcionario** que estende a classe **Pessoa** e possui um atributo **salario** e um **método aumentarSalario**, que aumenta o salário do funcionário de acordo com um percentual informado
- A classe **Funcionario** redefine o método **fazerAniversario** para, além de incrementar a idade e exibir uma mensagem, verificar se a idade é múltipla de 5 e, nesse caso, conceder um bônus ao funcionário

Atividade redefinição – Exemplo da execução

```
public class Main {  
    public static void main(String[] args) {  
        Funcionario funcionario1 = new Funcionario("João", 25, 3000.0);  
        funcionario1.fazerAniversario(); // Parabéns, João! Você agora tem 26 anos.  
        funcionario1.aumentarSalario(10); // O salário de João foi aumentado em 10%. Novo salário: R$ 3300.0  
        funcionario1.fazerAniversario(); // Parabéns, João! Você agora tem 27 anos.  
        funcionario1.fazerAniversario(); // Parabéns, João! Você agora tem 28 anos.  
        funcionario1.fazerAniversario(); // Parabéns, João! Você agora tem 29 anos.  
        funcionario1.fazerAniversario(); // Parabéns, João! Você agora tem 30 anos e ganhou um bônus de R$  
330.0.  
        funcionario1.aumentarSalario(5); // O salário de João foi aumentado em 5%. Novo salário: R$ 3465.0  
    }  
}
```


Entrega Atividade redefinição

- Entrega: <https://forms.gle/WM7F36KWz5aTuWsW9>

Links

- <https://netbeans.apache.org/download/nb17/index.html>
- <https://www.oracle.com/br/java/technologies/downloads/#jdk19-windows>
- <https://github.com/JoaoChoma/POO>

Referências utilizadas

- ASCENCIO, A. F. G. Fundamentos da programação de computadores. 2ª. Edição. São Paulo: Pearson Prentice Hall.
- <https://www.oracle.com/>

Programação Orientada a Objetos Aula 09

João Choma Neto
joao.choma@gmail.com